ICT Level 3

Lab Manual & Student Workbook

Level 3 - Classes 8, 9, 10

Level: Level 3

Class: Classes 8, 9, 10

Activities: 72 Lab Activities + 36 Dormitory Tasks

Curated and developed by

Hoysala T, Computer Science Teacher

MDRS SC-32 Bahaddurghatta (Kogunde), Chitradurga Taluk & District - 577519

Table of Contents

Chapter 1: Advanced Programming Concepts

Activity 1.1: Functions Practice

Activity 1.1: Functions Practice

Duration: 40 minutes

Resources Needed:
- Computer with Python 3 installed
- Text editor or IDE (Thonny, VS Code, or PyCharm)
- Student worksheet handout

Procedure: 1. Linked lesson: Level 3 Ch 1 2. Review the syntax for defining functions using `def` keyword. 3. Write a function `greet(name)` that returns a personalized greeting string. 4. Write a function `add(a, b)` that returns the sum of two numbers. 5. Write a function `is_even(n)` that returns `True` if n is even, `False` otherwise. 6. Write a function `factorial(n)` that computes n! using a loop. 7. Write a function `reverse_string(s)` that returns the reversed version of a string. 8. Test each function with at least three different inputs and record results. 9. Experiment with default parameter values and keyword arguments.

Questions: 1. What is the difference between a function parameter and a function argument? 2. Why is it important to use `return` instead of `print` inside a function? 3. Can a function call another function? Demonstrate with an example. 4. What happens if you forget the `return` statement? What value does the function return? 5. How do default parameters make functions more flexible?

Activity 1.2: Array Manipulation

Activity 1.2: Array Manipulation

Duration: 40 minutes

Resources Needed:
- Computer with Python 3 installed
- Text editor or IDE
- Sample data sets provided by teacher

Procedure: 1. Linked lesson: Level 3 Ch 1 2. Create a list of 10 integers and print the entire list. 3. Write code to find the maximum, minimum, and average of the list. 4. Use list slicing to extract the first 5 elements and the last 3 elements. 5. Append, insert, and remove elements from the list dynamically. 6. Sort the list in ascending and descending order using built-in methods. 7. Reverse the list without using the `reverse()` method (use a loop). 8. Create a 2D list (matrix) and write code to transpose it. 9. Merge two lists and remove any duplicate values.

Questions: 1. What is the time complexity of searching for an element in an unsorted list? 2. How does list slicing work? What does `list[2:7]` return? 3. What is the difference between `list.sort()` and `sorted(list)`? 4. How would you flatten a 2D list into a 1D list? 5. Why are lists mutable while strings are immutable in Python?

Chapter 2: Data Structures

Activity 2.1: Lists and Dictionaries

Activity 2.1: Lists and Dictionaries

Duration: 40 minutes

Resources Needed:
- Computer with Python 3 installed
- Text editor or IDE
- Dictionary data sample file

Procedure: 1. Linked lesson: Level 3 Ch 2 2. Create a dictionary storing a student record: name, age, grade, subjects. 3. Access, add, update, and delete key-value pairs in the dictionary. 4. Iterate over dictionary keys, values, and items using `for` loops. 5. Nest a list inside a dictionary (e.g., multiple grades per subject). 6. Nest a dictionary inside another dictionary (e.g., class roster). 7. Write a function that counts the frequency of each character in a string using a dictionary. 8. Convert a dictionary to a list of tuples and vice versa. 9. Use dictionary comprehension to create a squares dictionary `{x: x**2 for x in range(10)}`.

Questions: 1. What are the limitations of lists compared to dictionaries? 2. Why are dictionary keys required to be immutable? 3. How would you merge two dictionaries? What happens with duplicate keys? 4. When would you use a tuple instead of a list? 5. Explain the difference between `dict.get(key)` and `dict[key]`.

Activity 2.2: Data Organization Project

Activity 2.2: Data Organization Project

Duration: 40 minutes

Resources Needed:
- Computer with Python 3 installed
- Sample CSV data file (provided by teacher)
- Worksheet template

Procedure: 1. Linked lesson: Level 3 Ch 2 2. Read the provided CSV file containing student marks data. 3. Parse the data into a list of dictionaries, one per student. 4. Calculate each student's average mark and grade (A/B/C/D/F). 5. Organize students into a dictionary grouped by grade. 6. Find the top 3 students by total marks. 7. Identify the subject with the highest class average. 8. Generate a summary report displaying statistics per grade group. 9. Export the organized data to a new formatted text file.

Questions: 1. What challenges arise when working with real-world CSV data? 2. How would you handle missing or invalid data entries? 3. What are the advantages of organizing data into dictionaries over nested lists? 4. How could you extend this project to include data visualization? 5. Why is data cleaning an important step before analysis?

Chapter 3: Database Fundamentals

Activity 3.1: SQL Query Practice

Activity 3.1: SQL Query Practice

Duration: 40 minutes

Resources Needed:
- Computer with SQLite installed or online SQL editor
- Pre-built school database (students, teachers, courses tables)
- SQL reference sheet

Procedure: 1. Linked lesson: Level 3 Ch 3 2. Open the provided school database in SQLite. 3. Write a `SELECT` query to retrieve all students in Class 10. 4. Use `WHERE` clauses to filter students by marks range (e.g., above 80). 5. Use `ORDER BY` to sort results by name and by marks. 6. Use `GROUP BY` with aggregate functions: `COUNT`, `AVG`, `MAX`, `MIN`. 7. Write a query using `JOIN` to combine students and courses tables. 8. Use `LIKE` and wildcard operators to search for partial name matches. 9. Write a subquery to find students scoring above the class average.

Questions: 1. What is the difference between `WHERE` and `HAVING`? 2. Explain the purpose of primary keys and foreign keys in a relational database. 3. What type of JOIN returns only matching rows from both tables? 4. How would you find duplicate records in a table using SQL? 5. Why is indexing important for database query performance?

Activity 3.2: Database Creation

Activity 3.2: Database Creation

Duration: 40 minutes

Resources Needed:
- Computer with SQLite or MySQL installed
- Database design worksheet
- ER diagram template

Procedure: 1. Linked lesson: Level 3 Ch 3 2. Design an ER diagram for a library management system with tables: Books, Members, Loans. 3. Create the database and define each table with appropriate data types. 4. Set primary keys and foreign key constraints. 5. Insert at least 10 sample records into each table. 6. Write a query to check which books are currently on loan. 7. Write a query to find the most borrowed book. 8. Add a `CHECK` constraint to ensure loan dates are valid. 9. Export the database schema as a `.sql` file.

Questions: 1. What is the purpose of normalization in database design? 2. Explain the concept of referential integrity with a real-world example. 3. What are the trade-offs between using many small tables versus fewer large tables? 4. How would you design a database for an online bookstore? Sketch the ER diagram. 5. What is SQL injection and how can it be prevented?

Chapter 4: Web Technologies

Activity 4.1: HTML Page Creation

Activity 4.1: HTML Page Creation

Duration: 40 minutes

Resources Needed:
- Computer with a text editor
- Web browser (Chrome, Firefox, or Edge)
- HTML reference guide

Procedure: 1. Linked lesson: Level 3 Ch 4 2. Create a new file `myprofile.html` with proper `DOCTYPE` declaration. 3. Build the page structure: `<html>`, `<head>`, `<body>`. 4. Add a header with your name using `<h1>` and a subtitle using `<h2>`. 5. Create a paragraph introducing yourself with `<p>` tags. 6. Add an unordered list of your favorite subjects using `<ul>` and `<li>`. 7. Insert an image using `<img>` with appropriate `alt` text. 8. Create a table displaying your weekly timetable. 9. Add a navigation menu using

`<nav>` and anchor links `<a>`. 10. Include a contact form with input fields: name, email, and message. 11. Validate your HTML using the W3C validator.

Questions: 1. What is the difference between block-level and inline elements? Give examples. 2. Why is the `alt` attribute important for images? 3. What is semantic HTML and why is it preferred over `<div>`-only markup? 4. How do you create a hyperlink to an external website? 5. Explain the purpose of the `<meta>` tag in the `<head>` section.

Activity 4.2: CSS Styling Practice

Activity 4.2: CSS Styling Practice

Duration: 40 minutes

Resources Needed:
- Computer with a text editor
- Web browser
- The HTML page created in Activity 4.1
- CSS reference guide

Procedure: 1. Linked lesson: Level 3 Ch 4 2. Create a new file `styles.css` and link it to your HTML page. 3. Apply a global font family and background color to the `<body>`. 4. Style the header with centered text, custom colors, and padding. 5. Style the navigation menu as a horizontal bar with hover effects. 6. Add borders, padding, and margin to the timetable table. 7. Use class and ID selectors to style specific elements differently. 8. Create a simple responsive layout using CSS Flexbox for the main content. 9. Add a footer with background color and centered text. 10. Use pseudo-classes (`:hover`, `:focus`) for interactive elements. 11. Test the page on different screen sizes and adjust for responsiveness.

Questions: 1. What is the CSS box model? Name its four components. 2. What is the difference between `class` and `id` selectors in CSS? 3. How does Flexbox simplify page layout compared to float-based layouts? 4. What is a media query and how does it enable responsive design? 5. Explain the difference between `padding` and `margin`.

Chapter 5: Algorithms and Computational Thinking

Activity 5.1: Sorting Algorithm Visualization

Activity 5.1: Sorting Algorithm Visualization

Duration: 40 minutes

Resources Needed:
- Computer with Python 3 installed
- Python `turtle` module or `matplotlib`
- Sorting algorithm reference sheet

Procedure: 1. Linked lesson: Level 3 Ch 5 2. Write a Python function for Bubble Sort that prints each swap step. 3. Write a Python function for Selection Sort that prints each swap step. 4. Write a Python function for Insertion Sort that prints each swap step. 5. Create a list of 8 random integers between 1 and 50. 6. Run each sorting algorithm on the same list and count total swaps. 7. Visualize the sorting process

using bar charts (update after each swap). 8. Record the number of swaps for each algorithm in a comparison table. 9. Discuss which algorithm is most efficient for small vs. large datasets.

Questions: 1. What is the average time complexity of Bubble Sort? When is it acceptable to use? 2. How does Selection Sort differ from Insertion Sort in terms of swap count? 3. Why is $O(n^2)$ considered less efficient than O(n log n) for large datasets? 4. Can you think of a real-world scenario where a simple sorting algorithm is sufficient? 5. What is stable sorting and why does it matter?

Activity 5.2: Searching Algorithm Practice

Activity 5.2: Searching Algorithm Practice

Duration: 40 minutes

Resources Needed:
- Computer with Python 3 installed
- Text editor or IDE
- Search algorithm reference sheet

Procedure: 1. Linked lesson: Level 3 Ch 5 2. Implement Linear Search on a list of 20 sorted integers. 3. Implement Binary Search on the same sorted list. 4. Compare the number of steps each algorithm takes for various target values. 5. Test with best case (first element), worst case (last element), and middle case. 6. Record results in a table: algorithm, target, steps taken. 7. Apply Binary Search to find a word in a sorted dictionary file. 8. Discuss when Linear Search is preferred over Binary Search. 9. Write pseudocode for both algorithms before coding them.

Questions: 1. What precondition must be met before using Binary Search? 2. What is the time complexity of Linear Search vs. Binary Search? 3. Why is Binary Search not applicable to unsorted lists? 4. How would you modify Binary Search to find the first occurrence of a duplicate value? 5. Describe a scenario where Linear Search outperforms Binary Search in practice.

Chapter 6: Emerging Technologies

Activity 6.1: AI Demo Exploration

Activity 6.1: AI Demo Exploration

Duration: 40 minutes

Resources Needed:
- Computer with internet access
- Web browser
- AI exploration worksheet

Procedure: 1. Linked lesson: Level 3 Ch 6 2. Visit a pre-approved AI demo website (e.g., Google Teachable Machine, ChatGPT). 3. Explore text-based AI: ask it to explain a programming concept in simple terms. 4. Explore image recognition: upload images and observe classification results. 5. Record the AI's response accuracy for 5 different inputs. 6. Discuss how AI models are trained and what data they learn from. 7. Identify one AI application in healthcare, education, or transportation. 8. Write a short reflection on the ethical implications of AI. 9. Compare AI capabilities with human problem-solving abilities.

Questions: 1. What are the limitations of current AI systems? 2. How does machine learning differ from traditional programming? 3. What is bias in AI and why is it a concern? 4. Can AI replace human teachers? Discuss advantages and disadvantages. 5. How might AI change the job market in the next 10 years?

Activity 6.2: Cloud Storage Practice

Activity 6.2: Cloud Storage Practice

Duration: 40 minutes

Resources Needed:

- Computer with internet access
- Web browser
- Cloud storage account (Google Drive, OneDrive, or similar)
- Sample files for upload (document, image, spreadsheet)

Procedure: 1. Linked lesson: Level 3 Ch 6 2. Log in to your cloud storage account provided by the school. 3. Create a new folder structure: Subjects > ICT > Level 3. 4. Upload a document, an image, and a spreadsheet to the appropriate folders. 5. Share a file with a classmate and set viewing permissions. 6. Create a collaborative document and edit it simultaneously with a partner. 7. Download a file and verify its integrity matches the original. 8. Explore version history and restore a previous version of a document. 9. Discuss cloud storage vs. local storage: pros and cons.

Questions: 1. What happens to your files if you lose internet connectivity? 2. How does cloud storage protect your data from hardware failure? 3. What are the privacy risks of storing data on cloud platforms? 4. Explain the difference between SaaS, PaaS, and IaaS with examples. 5. Why do organizations use both cloud and local storage (hybrid approach)?

Dormitory Tasks

Task 1: Build a Calculator Program

Duration: 30 minutes

Create a Python calculator that supports addition, subtraction, multiplication, and division. Include input validation and a menu loop that lets the user perform multiple calculations until they choose to quit.

Task 2: Design a Student Database Table

Duration: 30 minutes

Write SQL statements to create a `students` table with columns: id, name, class, section, marks, and admission_date. Insert 15 sample records and write queries to find the top scorer, class averages, and students with marks below 50.

Task 3: Create a Personal Portfolio Webpage

Duration: 30 minutes

Build a complete HTML page with at least 5 sections: About Me, Education, Skills, Projects, and Contact. Include at least one image, a table, a list, and a contact form. Add basic CSS styling with custom colors and fonts.

Task 4: Write a Palindrome Checker

Duration: 30 minutes

Write a Python function `is_palindrome(text)` that checks if a given string is a palindrome (reads the same forwards and backwards). Ignore spaces and case. Test it with at least 10 different strings.

Task 5: Design a Book Database Schema

Duration: 30 minutes

Design and create an SQLite database for a library with tables: Books (book_id, title, author, genre, year), Members (member_id, name, join_date), and Loans (loan_id, book_id, member_id, loan_date, return_date). Write 5 useful queries.

Task 6: Build a Number Guessing Game

Duration: 30 minutes

Write a Python number guessing game where the computer generates a random number between 1 and 100. The player gets hints ('Too high!' or 'Too low!') and must guess within 7 attempts. Track and display the score across multiple rounds.

Task 7: Create an Interactive Quiz Webpage

Duration: 30 minutes

Build an HTML page with a 5-question multiple-choice quiz. Each question should have 4 options. Use JavaScript to check answers, display the score at the end, and provide feedback for each question. Style with CSS.

Task 8: Implement Bubble Sort with Complexity Analysis

Duration: 30 minutes

Write a Python implementation of Bubble Sort with an optimization flag to exit early if no swaps occur. Count and display the number of comparisons and swaps. Run it on lists of size 10, 50, and 100 and record the results.

Task 9: Build a To-Do List Application

Duration: 30 minutes

Create a Python to-do list program that allows users to add, remove, mark as complete, and view tasks. Store tasks in a list of dictionaries with fields: task description, priority (high/medium/low), and status (pending/completed).

Task 10: Design a Responsive Blog Layout

Duration: 30 minutes

Create an HTML/CSS blog page with a header, sidebar, main content area with 3 blog posts, and a footer. Use Flexbox for layout and media queries to make it responsive for mobile and desktop screens.

Task 11: Write a File Organizer Script

Duration: 30 minutes

Write a Python script that scans a directory and organizes files into folders based on their extensions (e.g., Images, Documents, Videos, Code). Log each move operation with timestamps to a file.

Task 12: Create a Class Attendance Database

Duration: 30 minutes

Design an SQLite database to track class attendance with tables: Students, Subjects, and Attendance (student_id, subject_id, date, status). Write queries to generate attendance reports, find students with below 75% attendance, and count absences per subject.

Capstone Projects

Capstone Project 1: School Management System

Activity CP-1: School Management System

Duration: Multiple sessions

Resources Needed:

- Computer with Python 3 and SQLite installed
- Project planning worksheet
- Database design template

Procedure: 1. Define the system requirements: manage students, teachers, subjects, and grades. 2. Design the database schema with at least 4 related tables. 3. Create the SQLite database and implement all tables with constraints. 4. Build a Python menu-driven interface for data entry and retrieval. 5. Implement CRUD operations (Create, Read, Update, Delete) for each entity. 6. Write reports: student grade sheets, teacher workload, class statistics. 7. Add input validation and error handling throughout the application. 8. Test the system with sample data and document all features. 9. Prepare a 5-minute presentation demonstrating the system.

Questions: 1. What challenges did you face when designing the database relationships? 2. How would you extend this system to support multiple schools? 3. What security measures would be needed for a real-world deployment? 4. How could this system be converted to a web application? 5. What additional features would parents and teachers find most useful?

Capstone Project 2: Interactive Learning Platform

Activity CP-2: Interactive Learning Platform

Duration: Multiple sessions

Resources Needed:

- Computer with web browser and text editor
- HTML, CSS, and JavaScript reference materials
- Sample quiz data in JSON format

Procedure: 1. Plan the platform structure: home page, subject pages, quiz section, and results. 2. Create the HTML structure for all pages with navigation. 3. Design a consistent CSS theme with custom colors and typography. 4. Implement a JavaScript quiz engine that loads questions from a JSON file. 5. Add a timer feature for each quiz question. 6. Track and display scores, progress, and performance statistics. 7. Create a leaderboard page that sorts scores in descending order. 8. Add accessibility features: keyboard navigation and screen reader support. 9. Test on multiple browsers and devices, then present to the class.

Questions: 1. How does loading quiz data from JSON make the platform more maintainable? 2. What accessibility standards should educational platforms follow? 3. How would you implement user authentication for the platform? 4. What metrics would help teachers understand student learning patterns? 5. How could this platform be integrated with a school's existing LMS?

Computer Club Activities

Computer Club Activity 1: Coding Challenge

Activity CC-1: Weekly Coding Challenge

Duration: 40 minutes

Resources Needed:

- Computer with Python 3 or JavaScript
- Online judge or local testing environment
- Challenge problem set

Procedure: 1. The teacher presents 3 coding problems of increasing difficulty. 2. Participants have 10 minutes to read and understand each problem. 3. Write a solution for the easiest problem (5 points). 4. Write a solution for the medium problem (10 points). 5. Attempt the hardest problem (20 points) for bonus credit. 6. Test solutions with the provided test cases. 7. Compare approaches with other participants after time is up. 8. Record scores and update the club leaderboard.

Questions: 1. What strategies did you use to break down complex problems? 2. How did your approach differ for easy vs. hard problems? 3. What debugging techniques helped you find errors quickly? 4. How can practicing coding challenges improve problem-solving skills? 5. What topics should the club focus on for the next challenge?

Computer Club Activity 2: Tech Talk Presentation

Activity CC-2: Tech Talk Presentation

Duration: 40 minutes

Resources Needed:

- Computer with presentation software
- Research materials and internet access
- Presentation evaluation rubric

Procedure: 1. Each participant selects a technology topic (AI, blockchain, quantum computing, etc.). 2. Research the topic using at least 3 credible sources. 3. Create a 3-5 slide presentation with key points and visuals. 4. Deliver a 3-minute presentation to the club. 5. Answer questions from the audience after the presentation. 6. Club members rate presentations on content, clarity, and visuals. 7. Discuss how the technology impacts daily life and future careers. 8. Select the best presentation for the school tech fair.

Questions: 1. How did researching this topic change your understanding of the technology? 2. What was the most surprising fact you discovered? 3. How can students stay updated with rapidly evolving technology? 4. What career opportunities exist in this technology field? 5. How might this technology change within the next 5 years?

Computer Club Activity 3: Debugging Bee

Activity CC-3: Debugging Bee Competition

Duration: 40 minutes

Resources Needed:
- Computer with Python 3 or JavaScript environment
- Pre-prepared buggy code files (10 programs with intentional errors)
- Scoring sheet

Procedure: 1. The teacher distributes 10 programs, each with 1-3 bugs. 2. Participants work individually to find and fix all bugs. 3. Each correctly fixed bug earns points (1 point per bug). 4. The first participant to fix a program earns bonus points. 5. After 30 minutes, reveal solutions and discuss each bug. 6. Award prizes to the top 3 scorers. 7. Discuss common bug categories: syntax, logic, and runtime errors. 8. Share debugging tips and tools that helped during the competition.

Questions: 1. What types of bugs were most common in the competition programs? 2. How do you distinguish between a syntax error and a logic error? 3. What tools or techniques help you find bugs faster? 4. How can writing clean code prevent bugs in the first place? 5. What is the value of code reviews in preventing bugs?

Computer Club Activity 4: Hackathon Prep Workshop

Activity CC-4: Hackathon Prep Workshop

Duration: 40 minutes

Resources Needed:
- Computer with internet access
- Hackathon planning templates
- Past hackathon project examples

Procedure: 1. Review what a hackathon is and how teams collaborate under time pressure. 2. Form teams of 3-4 members with complementary skills. 3. Brainstorm 5 project ideas and evaluate feasibility for each. 4. Select the best idea and create a project plan with milestones. 5. Divide responsibilities: frontend, backend, design, and presentation. 6. Practice rapid prototyping by building a minimal version in 20 minutes. 7. Prepare a 2-minute pitch for the project concept. 8. Discuss time management strategies for hackathon events.

Questions: 1. What makes a hackathon project stand out to judges? 2. How do you divide work effectively in a team under time pressure? 3. What is a Minimum Viable Product (MVP) and why is it important? 4. How can you prepare mentally and physically for a long hackathon? 5. What skills are most valuable in a hackathon team?

Computer Club Activity 5: Website Design Challenge

Activity CC-5: Website Design Challenge

Duration: 40 minutes

Resources Needed:
- Computer with text editor and web browser
- Design inspiration websites (Dribbble, Awwwards)
- HTML/CSS reference materials

Procedure: 1. The teacher announces a theme (e.g., 'School Event', 'Environmental Awareness'). 2. Each participant sketches a wireframe on paper (5 minutes). 3. Build a complete one-page website with HTML and CSS (25 minutes). 4. The website must include: header, at least 3 sections, images, and a footer. 5. Websites are displayed on the projector for judging. 6. Club members vote on design, creativity, and content quality. 7. Discuss design principles: whitespace, contrast, alignment, and hierarchy. 8. Award badges for Best Design, Most Creative, and Best Content.

Questions: 1. What design principles made the winning website stand out? 2. How does color choice affect the mood and readability of a website? 3. Why is mobile responsiveness important even for simple websites? 4. How can you make a website accessible to users with disabilities? 5. What role does content play alongside visual design?

Computer Club Activity 6: Open Source Contribution Day

Activity CC-6: Open Source Contribution Day

Duration: 40 minutes

Resources Needed:
- Computer with Git and GitHub account
- List of beginner-friendly open source projects
- Git and GitHub tutorial reference

Procedure: 1. Introduction to open source: what it is and why it matters. 2. Create or log in to a GitHub account. 3. Find a beginner-friendly issue labeled 'good first issue'. 4. Fork the repository and clone it to your local machine. 5. Make the required code change or documentation improvement. 6. Write a clear commit message describing the change. 7. Submit a pull request with a description of the contribution. 8. Discuss the contribution guidelines and code of conduct.

Questions: 1. Why do companies and individuals contribute to open source projects? 2. What makes a good pull request description? 3. How does contributing to open source benefit your learning and career? 4. What are the risks of using open source software in production? 5. How can students get started with open source contribution?